EECS 3311 Software Design – Fall 2026

StudyBuddy

An AI Study and Learning Agent

Andrew Manga
October 3, 2026

Project Overview

StudyBuddy is a desktop AI agent that helps students learn from their own course materials. Students upload their slides, notes and readings. The agent extracts their key concepts, answers questions with citations, generates flashcards and quizzes, grades answers, and tracks which topics the student has mastered. A conversational Study Coach can carry out multi-step study tasks on its own by planning and calling these capabilities as tools.

Problem and Motivation

Students in a typical term manage hundreds of pages of slides, notes and readings across several courses. Research on learning consistently shows that active recall (testing yourself) and spaced practice are far more effective than re-reading, yet most students default to re-reading because the alternatives take effort to set up:

- Creating practice material by hand is a slow process
- Students often do not know what they are having trouble with
- General-purpose chatbots underperform at these tasks

StudyBuddy addresses these problems by grounding all generated content in the student's own materials, measuring mastery from the student's answers, and turning that information into targeted practice.

Target Users

The primary demographic for StudyBuddy is **university/college students**, preparing for examinations in courses with substantial reading or lecture material.

The system is a single-user desktop application. Users are assumed to be comfortable with basic desktop software. A CLI alternative is available for more technically minded users.

Agent Description

StudyBuddy will excel in the following areas:

- Understanding materials. It can ingest and index course files, extract key concepts and answer questions with citations.
- Practicing. It can generate flashcards with spaced-repetition review, generate quizzes, grade answers and explain mistakes.
- Personalization. It can identify weak topics so that practice can focus on them.
- Acting autonomously. The Study Coach takes a high-level goal, decides which steps are needed, and carries them out by calling the capabilities above as tools.
- Reports. It will keep track of progress and the user’s study history.

AI/LLM Models

- The main model is **Google Gemini**, accessed through the Gemini API and integrated with LangChain4j's Gemini module. The low-cost Gemini Flash-Lite model is used for generation

and grading, and the higher-cost (and presumably smarter) Gemini Flash model handles the Study Coach's planning and tool calling.

- In testing, a **FakeLLMClient** returns scripted responses, so deterministic components can be tested without network access.
- All models sit behind StudyBuddy's own LLMClient interface, so another provider (for example, DeepSeek) can be added as a new adapter without changing the rest of the system. The API key is read from an environment variable, never committed to the online repository, and given to the tester privately.

Architecture

The system uses a layered architecture, comprised of five main layers.

Presentation Layer	A Swing GUI and a CLI with no business logic. Both call the same *StudyAgentFacade*, so every feature behaves identically from either interface.
Application Layer	*StudyAgentFacade* is the lone entry point. It delegates to one service per feature area (*MaterialService*, *QuizService*, *MasteryService,* etc.), which hold the business rules.
Agent Layer	The *AgentController* runs the Study Coach. Its *Planner* runs a bounded reason-act loop with the LLM. *ToolManager* validates and executes tool calls, and *ConversationMemory* and *AgentTrace* record steps. Each tool is a thin wrapper around an existing service, so the agent reuses the same tested code as the GUI.
AI Integration Layer	The *LLMClient* interface hides which model provider is in use. *PromptBuilder* assembles prompts from templates, and *ResponseParser* validates structured LLM output.
Domain & Persistence Layer	Plain domain objects (*Course*, *Material*, *Quiz*, *Question*, *Flashcard, TopicMastery*, etc.) and repositories that store them in a local SQLite database.

The design keeps the LLM behind clear boundaries so that its non-deterministic output cannot corrupt application state.

- Every LLM call goes through the *LLMClient* interface.
- The LLM can only request tool calls.
- Responses are parsed and checked.
- All objective grades, feedback, schedules, etc. are computed by code and not generated by the LLM.
- The reasoning loop has a step limit and timeouts, and every step is recorded.
- LLM calls run on background threads so that the GUI remains responsive.

Scope

- Single local user, no accounts or login.
- Materials are text-based PDF, TXT or Markdown files in English. Scanning documents is not allowed.
- An internet connection is required to reach the AI model.

Features

F01 — Course and Material Management

The student creates a course and uploads lecture slides, notes or readings (PDF, TXT or Markdown) in the Materials tab. The system extracts the text, splits it into page-tagged sections and indexes them for search, so later features can find and cite relevant passages. The material then appears in the course's list. Files with no readable text, such as scanned PDFs, are flagged instead of indexed, and duplicate uploads are detected. ***Classification:*** *Deterministic*.

F02 — Concept Extraction

From a selected material, the LLM identifies key concepts, each with a short definition, source pages and prerequisites. Duplicates are merged with the course's existing concepts, and the student reviews or edits the list before saving. This list becomes the course's topic map: questions and flashcards are tagged with concepts, and mastery is tracked per concept. Deleting a concept leaves its questions untagged rather than removing them. ***Classification:*** *AI-based*.

F03 — Question Answering over Course Materials

The student asks a question about a course in the Ask panel. The agent retrieves the most relevant passages from the course materials, and the LLM answers using only those passages, citing the material and page for each claim. Clicking a citation will open the source. If the materials do not cover the question, the agent says so instead of guessing or offers a clearly labelled general-knowledge answer. ***Classification:*** *Hybrid*.

F04 — Flashcard Generation and Review

The student chooses a material or set of concepts and a number of cards, and the LLM generates a deck that the student can edit before saving. In review mode, a spaced-repetition schedule decides which cards are due, and the student rates their recall of each card. As a result, the schedule and the concept's mastery record update. If the content cannot support the requested number of cards, fewer are generated and the student is told why. ***Classification:*** *Hybrid*.

F05 — Quiz Generation

The student picks the scope, number of questions, question types (multiple choice, true/false, short answer) and difficulty. The LLM generates questions from the retrieved material, each tagged with a concept, difficulty and source page, along with an answer key and, for short answers, a grading rubric. Every question is checked for validity (for example, exactly one correct option), and invalid ones are regenerated or dropped before the quiz opens. ***Classification:*** *AI-based*.

F06 — Quiz Grading

When the student submits a quiz, multiple-choice and true/false answers are graded against the answer key, while short answers are graded by the LLM against the rubric, with written feedback. The results view shows the score and per-question feedback, and results are saved for progress tracking. If AI grading fails, those answers are marked pending for later re-grading, and the student can dispute any AI-assigned grade. ***Classification:*** *Hybrid*.

F07 — Answer Explanation

From the quiz results, the student clicks Explain on any question. Given the question, the student's answer, the correct answer and the source passage, the LLM explains why the correct answer is right and what misconception likely led to the student's answer, citing the source. A follow-up question from the student is permitted. If the source material has since been deleted, the explanation relies on the answer key alone and says so. ***Classification:*** *AI-based*.

F08 — Weak-Topic Identification

From quiz results and flashcard ratings, the system computes a mastery score per concept, weighting recent and harder questions more heavily, and flags concepts below a threshold as weak. The Progress tab ranks concepts by mastery, with shortcuts to practise them. On request, the LLM reviews the student's mistakes for patterns, such as confusing two concepts, and suggests a focus. Concepts with too little data are not labelled weak. ***Classification:*** *Hybrid*.

F09 — Study Coach

In the Coach panel, the student states a goal in plain language, such as “I have a midterm Friday on chapters 3–5, help me prepare.” The agent plans the steps and carries them out by calling tools (checking weak topics, searching materials, generating a quiz or flashcards), showing each step as it runs. It remembers the conversation, asks for clarification when a request is ambiguous, stops after a fixed number of steps, and declines to complete graded work. ***Classification:*** *AI agent*.

F10 — Progress Tracking and Study History

The dashboard and Progress tab show the student's quiz score trend, mastery per concept, flashcards due and a filterable history of past activity. Views refresh automatically as new results are recorded, and a progress report can be exported. ***Classification:*** *Deterministic*.

UML Class Diagram

See /uml/ClassDiagram.svg.

Design Patterns

Included for each design pattern, there a list of participating classes and their roles as seen in the UML class diagram.

Facade

- StudyAgentFacade → the facade.
- MainWindow, the FeaturePanel subclasses and StudyAgentCli → clients.
- MaterialService, LearningContentService, FlashcardService, QuizService, MasteryService, ProgressService and AgentController → the subsystem.

Both the GUI and the CLI must offer every feature, but the logic is spread across six services and the agent. The facade gives one simple entry point per feature, so both interfaces call the same methods, behave identically and contain no business logic. Without it, each interface would depend on every service and repeat the same orchestration, so the GUI and CLI would gradually drift apart.

Strategy

- LLMClient → a strategy interface, implemented by GeminiClient and FakeLLMClient.
- RetrievalStrategy → a strategy interface, implemented by EmbeddingRetriever and KeywordRetriever.
- GradingStrategy → a strategy interface, implemented by RuleBasedGrader and LLMRubricGrader.
- The services, the Planner and QuizService → contexts.

Several behaviours have interchangeable algorithms (e.g. which model generates text, how passages are retrieved, how an answer is graded). Callers depend only on the interface, so switching provider, retrieval method or question type means adding a class rather than editing conditionals in every service. It also makes unit tests deterministic, because the fake client and keyword retriever can be swapped in.

Adapter

- LLMClient → the target interface.
- GeminiClient → the adapter.
- LangChain4j's ChatModel → the adaptee.
- The services and the Planner → clients.

StudyBuddy needs a small, provider-neutral interface, while LangChain4j has its own message and response types. GeminiClient converts between the two, so no library or provider classes leak outside the AI integration layer. Changing provider means writing a new adapter. Without one, every service would import library types, and a library upgrade or provider change would ripple through the whole system.

Command

- AgentTool → the command interface.
- SearchMaterialsTool, GetWeakTopicsTool, GenerateQuizTool and the other tools → concrete commands.
- The services they call → receivers.

- ToolManager → the invoker.
- Planner → the client, issuing the calls the LLM chooses.

The Study Coach picks actions at runtime from a tool name and arguments produced by the LLM. Wrapping each action as an object with a name, description, parameter schema and execute() method allows the ToolManager to handle everything in one place. New tools are added by registering a class. Without this, the agent would need an ever-growing switch statement mapping names to service calls, with validation and logging repeated in each branch.

Observer

- EventBus → the subject.
- StudyEventListener → the observer interface.
- MainWindow → the concrete observer, which refreshes its panels.
- QuizService and MaterialService → publish events through the EventBus.

When a quiz is graded or a material is uploaded, several views (e.g. dashboard, progress charts, weak-topic list) need to update. Services publish an event without knowing which views exist, so the application layer stays independent of Swing, and new views can subscribe without changing any service. Without it, services would need references to GUI panels (the wrong dependency direction), or the GUI would have to keep polling for changes.

Builder

- PromptBuilder → the builder.
- LLMRequest → the product.
- LearningContentService and Planner → clients.

Each prompt combines optional parts (system & task instructions, an output format, tool descriptions, etc.). Different features need different combinations. PromptBuilder assembles them step by step with chained calls ending in build(), so prompt formatting only resides in one place and every request is consistent. Without it, each feature would build prompts by its own string concatenation, which makes them inconsistent and hard to adjust when testing reveals a problem.

Factory

- QuestionFactory → the creator, through createQuestion(type, data).
- Question → the abstract product.
- MultipleChoiceQuestion, TrueFalseQuestion and ShortAnswerQuestion → concrete products.
- QuizService → the client.

The LLM returns quiz questions as structured data of different types. The factory turns each one into the right Question subclass, so QuizService works only with the abstract Question. Adding a question type means adding a subclass and one case in the factory. Without it, the code that builds each question type would be scattered through quiz generation, and anywhere saved quizzes are loaded.

UML Use-Case Diagram

See /uml/UseCaseDiagram.svg.

Use-Case Descriptions

UC01 — Manage Course Materials

The **Student** wants their course materials available to the agent, so they create a course and upload a PDF, TXT or Markdown file. The system extracts, splits and indexes the text, and the material appears in the course's list. If the file is unsupported, has no readable text or is a duplicate, the system reports the problem and does not index it.

- **Preconditions:** The application is running.
- **Postconditions:** The course and material are stored, and the material's text is indexed for search.

UC02 — Analyze Material

To understand a material quickly, the **Student** selects it and requests a list of its key concepts. The LLM Service extracts them from the material's text, the Student reviews the concepts before saving, and the confirmed concepts are stored. If the LLM Service is unavailable or returns malformed output, the request is retried and the Student is notified.

- **Preconditions:** The material has been uploaded and contains readable text.
- **Postconditions: The confirmed concepts are added** to the course's concept list.

UC03 — Ask a Question

The **Student** asks a question about a course to get an answer based on their own materials. The system retrieves the most relevant passages, and the **LLM Service** answers from them, citing material and page. If no relevant passages are found, the system says the materials don't cover the question rather than guessing, and can offer a clearly labelled general-knowledge answer instead.

- **Preconditions:** The course has at least one indexed material.
- **Postconditions:** The answer and its citations are displayed and saved to the question history.

UC04 — Consult Study Coach

The **Student** states a study goal in the Coach panel. Using the **LLM Service**, the agent plans the steps needed and runs tools to carry them out, such as checking weak topics, searching materials, or creating quizzes or flashcards. It then reports what it did, with links to anything it created. If the request is ambiguous, the agent asks a clarifying question. If a tool fails or the step limit is reached, it explains what was completed and what remains.

- **Preconditions:** A course is selected and has indexed materials.
- **Postconditions:** The response, any items created and the step trace are saved, and the conversation memory is updated.

UC05 — Study Flashcards

To memorize key concepts, the **Student** chooses a material or set of concepts and requests a deck. The **LLM Service** generates the cards, and the Student edits and saves them. The Student then reviews the cards that are due and rates how well they remembered each one, which reschedules it. If the content supports fewer cards than requested, the Student is told why. If no cards are due, the system shows the next due date.

- **Preconditions:** The course has an indexed material or extracted concepts.
- **Postconditions:** The deck is saved, and the cards' schedules are updated along with mastery evidence for their concepts.

UC06 — Take a Quiz

To test their knowledge, the **Student** picks the scope, question types and difficulty. The **LLM Service** generates questions, which the system validates. After the Student submits, the system grades objective answers by rule and short answers with the LLM Service, then shows the score and feedback. Invalid questions are regenerated or dropped. If AI grading fails, those answers are marked as pending for later grading. This use case can be extended by UC08.

- **Preconditions:** The course has indexed materials.
- **Postconditions:** The quiz and graded attempt are saved and recorded as mastery evidence.

UC07 — Review Progress

The **Student** opens the Progress tab to see how they're doing. The system assesses topic mastery (UC09) and shows score trends, weak topics and flashcards due. If there's no activity yet, it shows suggestions for getting started instead.

- **Preconditions:** A course is selected.
- **Postconditions:** No data is changed. A progress report file is saved if the Student exports one.

UC08 — Explain Answer

After a quiz is graded, the **Student** asks why an answer was wrong. The **LLM Service** explains the correct answer and the likely misconception, citing the source passage. If that source has since been deleted, the explanation relies on the answer key alone and says so.

- **Preconditions:** A quiz attempt has been graded, and the answer to be explained is not still pending.
- **Postconditions:** The explanation is saved with the attempt and displayed, so asking again shows it without calling the LLM Service.

UC09 — Assess Topic Mastery

The system computes a mastery score for each concept from quiz results and flashcard ratings, and flags weak topics. If the **Student** requests a diagnosis, the **LLM Service** looks for patterns in their recent mistakes and suggests what to focus on. Concepts with too little evidence are marked as such rather than labelled weak.

- **Preconditions:** None. With no graded history, all concepts are marked as having insufficient data.
- **Postconditions:** The mastery scores are updated and available to the use case that included this one.

UML Sequence Diagrams

See /uml/SD{01-06}_{SequenceName}.png.

Feature-to-Design Traceability Table

Feature	Description	Type	Related Use-Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01	Manage course materials	Deterministic	UC01 Manage Course Materials	MaterialsPanel, StudyAgentFacade, MaterialService, DocumentParser, RetrievalStrategy, Repository, EventBus, MainWindow	uploadMaterial(), extractPages(), splitIntoChunks(), index(), save(), publish(), onStudyEvent()	SD01	Facade, Strategy, Observer
F02	Extract key concepts	AI-based	UC02 Analyze Material	ConceptsPanel, StudyAgentFacade, LearningContentService, PromptBuilder, LLMClient, ResponseParser, Concept, Repository	extractConcepts(), build(), chat(), parse(), save()	SD02	Facade, Builder, Strategy, Adapter
F03	Answer questions from course materials	Hybrid	UC03 Ask a Question	AskPanel, StudyAgentFacade, LearningContentService, RetrievalStrategy, PromptBuilder, GeminiClient, ChatModel, ResponseParser	askQuestion(), answerQuestion(), retrieve(), withExcerpts(), build(), chat(), parse(), validateCitations()	SD02	Facade, Strategy, Builder, Adapter
F04	Generate and review flashcards	Hybrid	UC05 Study Flashcards	FlashcardPanel, StudyAgentFacade, FlashcardService, RetrievalStrategy, LLMClient, Flashcard, Repository	generateFlashcards(), generateCards(), retrieve(), chat(), getDueCards(), recordReview(), review(), save()	SD06	Facade, Strategy
F05	Generate quizzes	AI-based	UC06 Take a Quiz	QuizPanel, StudyAgentFacade, QuizService, RetrievalStrategy, LLMClient, QuestionFactory, Question, Repository	generateQuiz(), retrieve(), chat(), createQuestion(), isValid(), save()	SD03	Facade, Factory, Strategy

F06	Grade quiz answers	Hybrid	UC06 Take a Quiz	QuizPanel, StudyAgentFacade, QuizService, GradingStrategy, RuleBasedGrader, LLMRubricGrader, LLMClient, QuizAttempt, EventBus	submitQuiz(), submitAttempt(), grade(), chat(), submit(), save(), publish()	SD04	Strategy, Observer, Facade
F07	Explain answers	AI-based	UC08 Explain Answer	QuizPanel, StudyAgentFacade, LearningContentService, RetrievalStrategy, PromptBuilder, LLMClient, ResponseParser	explainAnswer(), retrieve(), build(), chat(), parse()	SD02	Facade, Builder, Adapter
F08	Identify weak topics	Hybrid	UC09 Assess Topic Mastery, UC07 Review Progress	ProgressPanel, StudyAgentFacade, MasteryService, Repository, TopicMastery, LLMClient	getWeakTopics(), computeMastery(), findByCourse(), diagnose()	SD05	Facade
F09	Study Coach agent	AI agent	UC04 Consult Study Coach	CoachPanel, StudyAgentFacade, AgentController, ConversationMemory, Planner, LLMClient, ToolManager, AgentTool, GetWeakTopicsTool, AgentTrace	askCoach(), handleMessage(), addTurn(), run(), buildPrompt(), chat(), getToolSpecs(), execute(), validateArgs(), recordStep()	SD05	Command, Facade, Strategy
F10	Track progress and study history	Deterministic	UC07 Review Progress	ProgressPanel, MainWindow, EventBus, StudyAgentFacade, ProgressService, Repository	publish(), onStudyEvent(), refresh(), getProgress(), getHistory(), exportReport()	SD04	Observer, Facade

Feature Implementation Explanations

F01 — Course and Material Management

When the student uploads a file, the facade passes it to MaterialService, which uses DocumentParser to extract the text page by page. Then, the service splits the text into page-tagged chunks, indexes them through the RetrievalStrategy and saves the material. Finally, it publishes an event on the EventBus so MainWindow refreshes the materials list. If no readable text is found, the service raises an error, which the panel shows instead.

F02 — Concept Extraction

LearningContentService builds a concept-extraction prompt from the material's chunks and sends it through LLMClient, and ResponseParser turns the reply into candidate Concept objects. The service merges duplicates with the course's existing concepts and returns the list for the student to confirm. Once saved, these concepts become the tags used by quizzes, flashcards and mastery tracking.

F03 — Question Answering over Course Materials

LearningContentService first asks the RetrievalStrategy for the passages most relevant to the question. If none are relevant enough, it returns a "not covered" answer. Otherwise, PromptBuilder packages the numbered passages with the question, and GeminiClient converts the request for LangChain4j's ChatModel, which calls the LLM service. ResponseParser then parses the answer and removes any citation that doesn't match a passage it was given.

F04 — Flashcard Generation and Review

For generation, FlashcardService retrieves passages for the chosen concepts, asks the LLM for cards, and saves each valid one. For review, the service loads the cards due today. Each rating goes to Flashcard.review(), which moves the card between spaced-repetition boxes and sets its next due date, and the updated card is saved.

F05 — Quiz Generation

QuizService retrieves passages for the quiz's scope, then asks the LLM for each question at the required type and difficulty. QuestionFactory turns each reply into the right Question subclass, and isValid() checks it. Invalid questions are regenerated up to twice or dropped. The finished Quiz is saved and opened in the panel.

F06 — Quiz Grading

QuizService.submitAttempt() picks a grading strategy for each answer. RuleBasedGrader compares multiple-choice and true/false answers with the answer key, while LLMRubricGrader asks the LLM to score short answers against the rubric. The attempt is then marked submitted and saved, and a QuizGraded event is published so progress views refresh.

F07 — Answer Explanation

The facade passes the question and the student's answer to LearningContentService.explainAnswer(). The service retrieves the question's source passage, builds an explanation prompt and calls the LLM. The parsed explanation is cached with the attempt, so asking again doesn't call the model a second time.

F08 — Weak-Topic Identification

MasteryService.computeMastery() loads the course's graded answers and flashcard ratings and computes a score for each concept, giving more weight to recent and harder questions. Concepts below the threshold are flagged as weak. getWeakTopics() returns these ranked, and diagnose() can ask the LLM to describe patterns in recent mistakes.

F09 — Study Coach

AgentController records the message in ConversationMemory, creates an AgentTrace and hands the goal to the Planner. At each step, the Planner builds a prompt from the conversation and the tool descriptions provided by ToolManager, and calls the LLM. If the reply requests a tool, ToolManager checks the arguments, runs the matching AgentTool (which calls an existing service

such as MasteryService) and records the step. The loop ends when the LLM gives a final answer or after eight steps, and the reply is saved to memory.

F10 — Progress Tracking and Study History

Whenever a service records new activity, it publishes an event on the EventBus. MainWindow receives the event and tells the progress panel to refresh. The panel requests a progress report from the facade, and ProgressService builds it from the stored quiz attempts and flashcard reviews. The report can also be exported to a file.